

RefactorTactics: Product Requirements Document e piano completo di sviluppo

Executive summary

RefactorTactics è un gioco tattico a turni simultanei nel quale ogni giocatore pianifica movimento e azioni senza conoscere il piano avversario. Gli alleati, invece, vedono in tempo reale percorsi, bersagli, aree d'effetto e intenzioni strategiche dei compagni. La risoluzione avviene contemporaneamente secondo regole deterministiche e verificabili.

La caratteristica distintiva non è soltanto il combattimento, ma la **mappa sistemica multilivello**. Le celle rappresentano superfici, altezza, copertura, visibilità, pericoli, collegamenti verticali, condizioni ambientali e interazioni. Il pathfinding non cerca semplicemente la strada più corta: interpreta un grafo semantico in base al personaggio, agli effetti attivi e allo stato corrente della mappa.

L'architettura raccomandata è:

Strato	Tecnologia principale
Regole, pathfinding, risoluzione, networking	Unreal C++
Presentazione, authoring, animazioni, prototipi	Blueprint
Abilità e status	Gameplay Ability System
Classificazione semantica	Gameplay Tags
Contenuti interni	Data Asset e Primary Data Asset
Mod pubbliche	JSON dichiarativo validato
Modalità di gioco	Game Feature plugin
Interfaccia	UMG, Common UI, Enhanced Input
Multiplayer	Server autoritativo con replica filtrata
Test e profiling	Automation Framework, Functional Tests, Unreal Insights

Alla data del **1° agosto 2026**, la baseline consigliata è **Unreal Engine 5.8**, pubblicato il 23 giugno 2026 e indicato da Epic come l'ultima major release pianificata della generazione UE5. Il progetto deve bloccare una versione precisa del motore e aggiornarla soltanto dopo una build stabile e una suite di regressione completa. ¹

La strategia di linguaggio è **C++ più Blueprint**, non C# come dipendenza di produzione. Epic documenta esplicitamente che Blueprint e C++ sono pensati per essere combinati: C++ offre una base

rigorosa, testabile e performante, mentre Blueprint consente iterazione visuale e specializzazione dei sistemi esposti dal codice. ²

Il percorso di prodotto è diviso in quattro grandi risultati:

Risultato	Prodotto ottenuto
Sandbox didattica	Camera, selezione, griglia e pedine
Vertical slice locale	Turni simultanei, mappa multilivello, abilità e interfaccia
Prodotto multiplayer	Match competitivo, personalizzazione, IA e servizi online
Piattaforma estendibile	Mod support e successiva espansione roguelike deckbuilding

Deliverable stampabili

Sono disponibili tre versioni dello stesso documento:

- [Scarica il PRD stampabile in PDF](#)
- [Scarica il PRD modificabile in DOCX](#)
- [Scarica il sorgente Markdown](#)

Il PDF usa formato pagina standard, margini da stampa, tabelle senza sfondi scuri, diagrammi testuali e numeri di pagina. Il DOCX è pensato per aggiungere note, decisioni e revisioni del progetto.

Product Requirements Document

Identità del prodotto

Nome di lavoro: RefactorTactics

Genere: tattico multiplayer a turni simultanei

Motore: Unreal Engine

Modalità principale: competitivo a squadre

Espansione futura: cooperativa roguelike con deckbuilding

Principio centrale: la mappa non è uno sfondo, ma un sistema di gioco.

La promessa al giocatore è:

Ogni turno è un problema tattico condiviso: leggi una mappa viva, comunichi il tuo intento agli alleati, prevedi il nemico e osservi la risoluzione simultanea delle decisioni.

Assunzioni e decisioni ancora aperte

Le informazioni non specificate vengono mantenute come decisioni formali, anziché nasconderle dietro assunzioni implicite.

Area	Baseline proposta	Stato
Piattaforma iniziale	PC Windows	Da confermare
Store	Steam o Epic Games Store	Da decidere

Area	Baseline proposta	Stato
Formato completo	Quattro contro quattro	Proposta
Formato prototipo	Uno contro uno, poi due contro due	Raccomandato
Co-op roguelike	Da uno a quattro giocatori	Proposta
Durata match competitivo	Venti-trentacinque minuti	Da validare
Durata fase di pianificazione	Quaranta-sessanta secondi configurabili	Da validare
Monetizzazione	Non specificata	Aperta
Budget	Non specificato	Aperto
Team	Solo developer o micro-team	Assunzione di roadmap
Direzione artistica	Non specificata	Aperta
Rating	Non specificato	Aperto
Localizzazione	Italiano e inglese iniziali	Proposta
Backend online	Astrazione compatibile con EOS o Steam	Da decidere
Matchmaking classificato	Successivo alla closed beta	Proposta
Fog of war	Non ancora definita	Decisione di game design

La roadmap temporale presentata più avanti è aggressiva per uno sviluppatore alle prime armi. È plausibile per un micro-team o per sviluppo quasi full-time; lavorando part-time e imparando contemporaneamente, è ragionevole considerare un orizzonte più lungo.

Obiettivi del prodotto

RefactorTactics deve:

1. offrire decisioni simultanee comprensibili e non arbitrarie;
2. rendere la comunicazione tra alleati parte naturale dell'interfaccia;
3. costruire mappe che modificano movimento, tiro, visibilità e abilità;
4. consentire build differenti senza introdurre progressione competitiva pay-to-win;
5. risolvere ogni turno in maniera deterministica;
6. separare regole, dati e presentazione;
7. essere predisposto per le mod senza dipenderne durante lo sviluppo iniziale;
8. riutilizzare il core competitivo nella futura modalità roguelike;
9. funzionare come percorso pratico per imparare Unreal Engine, Blueprint e C++.

Utenti principali

Utente	Bisogno
Giocatore competitivo	Decisioni leggibili, equilibrio, counterplay e assenza di informazioni illegittime
Gruppo di amici	Pianificazione collaborativa anche senza chat vocale
Giocatore tattico single-player	IA credibile e strumenti per comprendere gli errori
Giocatore co-op	Sinergie, progressione della run e decisioni condivise
Creatore di mappe	Validator, anteprima del grafo e strumenti di test
Modder	Schema documentato, errori chiari e contenuti caricabili senza ricompilazione
Spettatore	Risoluzione leggibile, replay e visualizzazione degli intenti autorizzati

Loop competitivo

Il loop completo di un round è:

```
STATO PUBBLICO DELLA MAPPA
|
v
PIANIFICAZIONE PRIVATA
  Movimento + azione + reazione opzionale
  |
  +-----> Gli alleati vedono gli intenti
  |
  +-----> I nemici non ricevono i piani
  |
  v
CONFERMA O SCADENZA TIMER
|
v
VALIDAZIONE SERVER
|
v
RISOLUZIONE SIMULTANEA
|
v
EFFETTI AMBIENTALI E OBIETTIVI
|
v
NUOVO ROUND
```

Ogni personaggio dispone, nella baseline proposta, di:

Elemento	Quantità per round
Percorso di movimento	Uno
Azione principale	Una
Reazione o stance	Zero o una
Interazione ambientale	Integrata nell'azione o nel movimento
Modifica del piano	Illimitata fino al lock
Intent label	Una facoltativa

L'intento può essere etichettato come **Focus**, **Protezione**, **Scout**, **Controllo**, **Fuga**, **Trappola** o **Attesa**. L'etichetta comunica il motivo della scelta, non soltanto la geometria.

Modalità competitiva iniziale

Per evitare che il tutorial debba risolvere subito tutti i problemi di un'arena completa, la modalità iniziale dovrebbe essere **Relay Control**:

Regola	Baseline
Squadre tutorial	Una contro una, poi due contro due
Squadre prodotto	Quattro contro quattro
Obiettivo	Controllare un relay attivo al termine del round
Rotazione	Il relay cambia posizione ogni due round
Vittoria	Punteggio obiettivo o vantaggio alla fine del limite round
Knockout	Rientro dopo un periodo configurabile
Durata massima	Dodici round, da bilanciare
Funzione della mappa	Il relay può attivare ascensori, porte o pericoli

Questa modalità costringe i giocatori a muoversi, sfruttare la verticalità e interagire con le celle, invece di trasformare ogni partita in uno scontro statico.

Feature list consolidata

Priorità	Feature	Descrizione
Fondamentale	Turni simultanei	Pianificazione, lock, risoluzione e aftermath
Fondamentale	Planning privato	I nemici non ricevono i piani prima della risoluzione
Fondamentale	Intenti alleati	Percorso, target, area, stato e motivazione condivisi
Fondamentale	Griglia multilivello	Coordinate orizzontali più layer e archi verticali
Fondamentale	Pathfinding semantico	Costi dipendenti da unità, terreno, status ed effetti

Priorità	Feature	Descrizione
Fondamentale	Mappa dinamica	Celle e collegamenti modificabili durante la partita
Fondamentale	Linea di vista	Servizio separato da movimento e pathfinding
Fondamentale	Combattimento	Danno, cure, spostamenti forzati, status e reazioni
Fondamentale	Server authority	Il server valida e risolve tutte le azioni
Importante	Personalizzazione	Specializzazioni, moduli abilità, gadget e tratti
Importante	Gameplay Tags	Vocabolario comune per superfici, unità ed effetti
Importante	IA tattica	Planning simultaneo senza accesso ai piani privati umani
Importante	Editor mappe	Authoring, connettività, heatmap e validator
Importante	Replay	Log degli eventi deterministici e riproduzione
Importante	Accessibilità	Forme, pattern, scaling e controlli rimappabili
Importante	Analytics	Hook per UX, bilanciamento, rete e performance
Successiva	Spettatore	Viste autorizzate e timeline delle azioni
Successiva	Matchmaking	Sessioni, lobby, classificato e riconnessione
Finale del core	Mod support	JSON, pacchetti di contenuto e manifest
Espansione	Co-op roguelike	Run, incontri, boss, ricompense e meta-progression
Espansione	Deckbuilding	Carte tattiche, draft, reliquie e trasformazioni build

Personalizzazione dei personaggi

Ogni personaggio conserva una forte identità riconoscibile ma ammette varianti orizzontali.

PERSONAGGIO

- └─ Chassis fisso
 - | └─ statistiche base
 - | └─ silhouette
 - | └─ ruolo narrativo
 - | └─ affinità ambientale
- └─ Specializzazione
- └─ Modificatori abilità
- └─ Gadget
- └─ Tratti
- └─ Ultimate

Configurazione proposta per una build competitiva:

Slot	Scelta
Chassis	Fisso
Specializzazione	Una tra tre
Modificatore abilità	Due
Gadget	Due
Tratto	Due entro un budget
Ultimate	Una tra due o tre
Cosmetici	Liberi, senza impatto sulle regole

Un modificatore non deve essere semplicemente “più cinque per cento di danno”. Deve cambiare l’uso tattico.

Abilità base	Variante
Granata	Esplosione immediata
Granata	Fumo senza danno
Granata	Mina ritardata
Granata	Tre cariche minori
Dash	Attraversa una cella pericolosa
Dash	Lascia una scia
Dash	Scambia posizione con un alleato
Scudo	Protezione frontale
Scudo	Cupola statica
Scudo	Riflette il primo proiettile

Il Gameplay Ability System è adatto a organizzare abilità, costi, cooldown, attributi ed effetti; Gameplay Tags fornisce una tassonomia gerarchica utilizzabile per classificare e filtrare unità, superfici, status e requisiti. Epic usa questi sistemi anche nei propri sample, incluso Lyra. ³

Requisiti non funzionali

Area	Criterio di accettazione
Determinismo	Stesso snapshot, seed e piani producono lo stesso hash finale
Privacy	Un client nemico non riceve strutture contenenti il piano avversario
Prestazioni path preview	Risposta tipica entro un frame interattivo
Prestazioni UI	Modifica del piano senza hitch percepibili
Networking	Riconnessione e ricostruzione dello stato del round

Area	Criterio di accettazione
Testabilità	Resolver eseguibile senza caricare un livello grafico
Manutenibilità	Le regole non dipendono dai Widget
Scalabilità	PvP e roguelike come moduli separati
Accessibilità	Nessuna informazione critica affidata soltanto al colore
Modding	Dati invalidi rifiutati prima del match
Replay	Hash dello stato verificato dopo ogni fase
Salvataggi	Versione schema e migrazioni esplicite

Metriche di prodotto

La north-star proposta è la **collaborazione prodotta dagli intenti condivisi**:

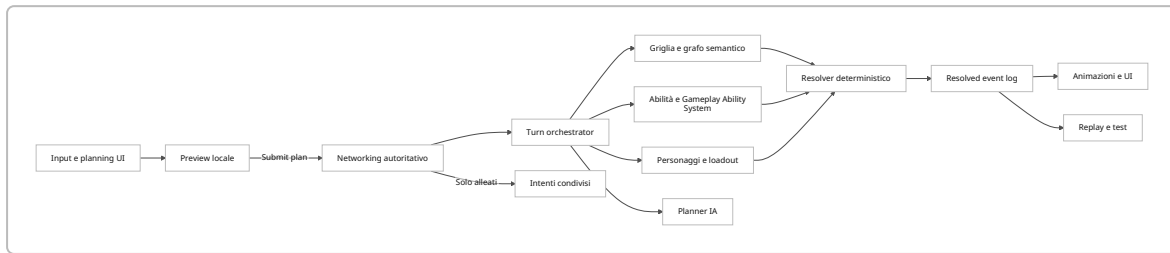
Percentuale di round nei quali un giocatore modifica il piano dopo aver visualizzato il piano di un alleato, senza aumento di conflitti o timeout.

Metriche complementari:

Categoria	Evento o misura
Onboarding	Tempo al primo percorso valido
Planning	Numero di revisioni piano
Collaborazione	Intenti visualizzati e modifiche successive
Conflitti	Collisioni alleate previste e risolte
UX	Annullamenti, errori di target e timeout
Bilanciamento	Pick rate, win-rate e impatto delle build
Mappa	Celle attraversate, zone ignorate e choke point
Rete	Latenza submit, perdita pacchetti e riconnessioni
Determinismo	Divergenze hash
Roguelike	Carte scelte, rimosse e trasformate
Mod	Errori schema, dipendenze e incompatibilità

Architettura dei sistemi e modelli dati

Architettura generale



Versione adatta alla stampa:

CLIENT

Input + Planning UI

|

v

Preview locale

|

ServerSubmitPlan

v

SERVER AUTORITATIVO

Networking

|

v

Turn Orchestrator

|

+---- Grid e Pathfinding

+---- Abilità e Status

+---- Personaggi

+---- IA

|

v

Resolver deterministico

|

+---- Intenti sanificati solo agli alleati

+---- Stato pubblico a tutti

+---- Event log di risoluzione a tutti

|

v

Replay + Test + Analytics

La separazione segue il Gameplay Framework di Unreal: `GameMode` vive soltanto sul server, mentre `GameState` rappresenta stato pubblico replicato; `PlayerState` contiene informazioni del giocatore che devono sopravvivere alla sostituzione del Pawn. Questo rende essenziale non inserire il piano privato nel `GameState`.

Moduli di codice

Modulo	Responsabilità
RTCore	Identificatori, tag, risultati, errori e tipi condivisi
RTGrid	Celle, chunk, coordinate e conversioni world-grid
RTPathfinding	Grafo, A*, provider di costo e cache
RTTurn	Stato round, validazione, lock e risoluzione
RTAbilities	Abilità, effetti, targeting e status
RTCharacters	Chassis, moduli, loadout e affinità
RTNetworking	RPC, replica, privacy e riconnessione
RTUI	HUD, preview, timeline e feedback
RTAI	Generazione candidati e utility
RTAnalytics	Eventi, metriche e adapter provider
RTModRuntime	Manifest, schema, caricamento e registrazione
RTEditor	Tool mappe, validator, heatmap e simulazione

Le modalità dovrebbero essere plugin separati:

Plugins

- RTMode_PvP
- RTMode_Roguelike
- RTContent_Core
- RTContent_Maps
- RTModRuntime

Game Features e Modular Gameplay sono stati progettati da Epic proprio per incapsulare feature attivabili e disattivabili; i Data Registries possono inoltre integrare dati provenienti dalle Game Features.

5

Modelli fondamentali

Tipo	Campi principali	Scopo
FRTGridCellId	X, Y, Layer	Identità stabile della cella
FRTCellStaticData	terreno, quota, cover, tag	Dati immutabili della mappa
FRTCellRuntimeState	effetti, occupazione, revision	Stato dinamico
FRTTraversalEdge	origine, destinazione, tipo, costo	Collegamento tra celle

Tipo	Campi principali	Scopo
<code>FRTTraversalProfile</code>	capacità, immunità, regole costo	Interpretazione della mappa da parte dell'unità
<code>FRTPathRequest</code>	unità, partenza, arrivo, snapshot	Richiesta deterministica
<code>FRTPlannedAction</code>	tipo, target, abilità, parametri	Singola parte del piano
<code>FRTPlannedTurn</code>	giocatore, revisione, azioni	Piano privato completo
<code>FRTTeamIntentView</code>	geometria e label sanificate	Informazione per gli alleati
<code>FRTResolvedEvent</code>	fase, ordine, sorgente, payload	Risultato atomico
<code>FRTModManifest</code>	ID, versione, dipendenze, hash	Compatibilità mod
<code>FTRunState</code>	nodo, seed, mazzi, reliquie	Stato roguelike

Sistema di celle

Una decisione tecnica importante è non trasformare ogni cella in un Actor con molti `UActorComponent`.

Il termine “componente della cella” deve descrivere un **frammento dati**, non necessariamente un componente Unreal allocato separatamente. Una mappa con migliaia di celle e decine di UObject per cella avrebbe costi inutili di memoria, garbage collection e aggiornamento.

Struttura proposta:

```

MAP ASSET
├─ Chunk
│   ├── Static cell array
│   ├── Traversal edges
│   ├── Cover edges
│   └─ Render instances
├─ Sparse runtime overrides
├─ Environment systems
└─ Graph revision

```

Frammenti semantici:

Frammento	Dati
Movimento	costo base, blocchi, direzioni
Superficie	acqua, ghiaccio, metallo, vegetazione
Altezza	quota, gradino, caduta

Frammento	Dati
Copertura	lato, altezza e distruttibilità
Visibilità	opacità, fumo, luce
Pericolo	fuoco, veleno, elettricità
Interazione	porte, terminali, leve
Trigger	ingresso, uscita, fine round
Suono	rumore prodotto e propagazione
Meteo	vento, pioggia, temperatura

I dati statici possono vivere in array contigui organizzati per chunk. Gli effetti temporanei vengono registrati come override sparsi. La geometria ripetuta dovrebbe essere visualizzata tramite instancing, lasciando gli Actor alle sole entità realmente interattive.

Grafo semantico multilivello

La cella è un nodo. Scale, rampe, portali, ascensori e salti sono archi.

```

FTraversalEdge
├─ From
├─ To
├─ EdgeType
├─ BaseCost
├─ RequiredTags
├─ BlockedTags
├─ Capacity
└─ RuntimeState

```

Esempio:

```

Cella (10, 5, Piano 0)
  |
  | Scala
  | Costo 2
  | Richiede Unit.CanClimb
  v
Cella (10, 5, Piano 1)

```

Un ascensore potrebbe avere costo basso ma richiedere energia. Un portale può collegare coordinate lontane. Un ponte distrutto disabilita l'arco. Un jump pad crea un arco temporaneo.

Formula del costo

Il pathfinder non deve conoscere direttamente lava, fumo o robot. Interroga provider di costo.

```
TraversalCost =  
    EdgeBaseCost  
    + TerrainCost  
    + UnitProfileCost  
    + StatusCost  
    + DynamicHazardCost  
    + ReservationCost  
    + ScenarioCost
```

Ogni provider può:

1. accettare l'attraversamento;
2. rifiutarlo;
3. aggiungere un costo intero;
4. aggiungere una conseguenza prevista;
5. indicare che il risultato dipende dalla revisione corrente.

Esempio:

```
Unità umana  
Acqua      +2  
Fuoco      bloccato  
Scala      +1  
  
Unità robotica  
Acqua      bloccato se elettrificata  
Fuoco      +0  
Scala      +2  
  
Unità volante  
Acqua      +0  
Fuoco      +1  
Scala      ignorata
```

Strategia A*

L'algoritmo raccomandato è A* custom su grafo esplicito.

```
OpenSet  
ClosedSet  
CostSoFar  
Parent  
Heuristic  
StableTieBreak
```

La funzione euristica deve restare ammissibile per il percorso di movimento. Una baseline è:

H =
distanza Manhattan orizzontale
+ numero minimo stimato di cambi layer

Le regole tattiche come esposizione o qualità della copertura **non devono alterare il percorso mostrato automaticamente al giocatore**, altrimenti la UI può sembrare imprevedibile. Devono invece essere usate dall'IA o da un comando esplicito come "percorso più sicuro".

Ricerca	Ottimizza
Path preview giocatore	Legalità e costo movimento
Path sicuro opzionale	Movimento più rischio
IA offensiva	Posizione, tiro, obiettivo
IA difensiva	Copertura, distanza, fuga
Map validator	Connettività e requisiti

Aggiornamenti dinamici

Ogni modifica topologica incrementa una `GraphRevision`.

```
Ponte distrutto
-> edge disabilitato
-> GraphRevision + 1
-> invalidazione path interessati

Portale creato
-> nuovo edge
-> GraphRevision + 1

Cella incendiata
-> aggiornamento runtime state
-> CostRevision + 1
```

Una cache deve includere:

```
Start
Goal
TraversalProfileId
GraphRevision
CostRevision
Path
```

Non è necessario ricalcolare tutta la mappa quando cambia una sola cella. I primi tutorial possono invalidare l'intera cache; l'invalidazione selettiva arriverà soltanto dopo il profiling.

Linea di vista e tiro

La linea di vista è un servizio separato dal pathfinding:

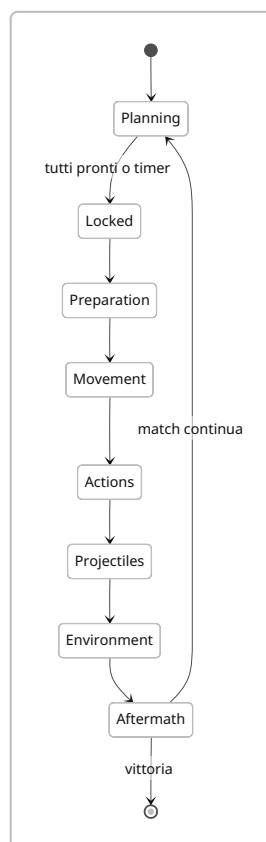
```
Movement Graph  
Line-of-Sight Service  
Projectile Service  
Cover Service  
Visibility Service
```

Un'abilità interroga:

```
CanTarget(Source, Target)  
GetCover(Source, Target)  
TraceProjectile(Path)  
GetVisibilityModifier(Source, Target)
```

Questa separazione consente a una cella di essere attraversabile ma opaca, oppure non attraversabile ma trasparente.

Risoluzione simultanea



Versione stampabile:

```

PLANNING
  Modifica piano, preview, undo, intenti alleati
    |
    v
LOCK
  Snapshot e validazione finale
    |
    v
PREPARATION
  Stance, scudi, reazioni preventive
    |
    v
MOVEMENT
  Microstep e conflitti
    |
    v
ACTIONS
  Abilità e interazioni
    |
    v
PROJECTILES
  Impatti, spostamenti, danni e status
    |
    v
ENVIRONMENT
  Fuoco, acqua, elettricità, trigger
    |
    v
AFTERMATH
  Obiettivi, KO, vittoria e nuovo round

```

Il resolver deve produrre una sequenza di eventi, non comandare direttamente animazioni.

```

FResolvedEvent
├─ Phase
├─ StableOrder
├─ SourceEntity
├─ TargetEntities
├─ TargetCells
├─ EffectId
├─ NumericPayload
└─ ResultingStateHash

```

L'animazione riproduce questi eventi. Il replay salva questi eventi. I test confrontano questi eventi.

Conflitti di movimento

Regole iniziali proposte:

Situazione	Risultato
Una unità entra in una cella libera	Movimento riuscito
Due unità vogliono la stessa cella	Vince la priorità deterministica
Due unità alleate si scambiano	Permesso solo se la regola di swap è attiva
Due nemici si scambiano	Bloccati o gestiti da regola esplicita
Un bersaglio viene spinto	Risoluzione nella fase di impatto
La cella diventa invalida	L'unità resta nell'ultima posizione valida
Un arco viene distrutto durante il turno	Gli step successivi che lo usano falliscono
Una unità viene eliminata	Libera la cella nella fase stabilita

La priorità deve usare valori deterministici:

```
PhasePriority
Initiative
ActionPriority
StableEntityId
```

Non usare l'ordine casuale di iterazione di un container.

Privacy della pianificazione

Flusso raccomandato:

```
Client locale
  crea preview
    |
    v
ServerSubmitPlan(Plan, Revision)
    |
    v
Server valida e conserva FPlannedTurn
    |
    +--> genera FTeamIntentView
    |    |
    |    +--> replica soltanto agli alleati
    |
    +--> conserva il piano completo soltanto sul server
```

La replica delle proprietà in Unreal è server-to-client; il server resta la fonte autorevole e i client non devono modificare direttamente lo stato replicato. ⁶

Implementazione proposta:

1. `ARTPlayerController` invia `ServerSubmitPlan`.
2. `ARTTurnSubsystem` valida il piano.
3. Il piano completo rimane in memoria server.
4. `ARTTeamPlanningChannel`, uno per squadra, contiene soltanto intenti sanificati.
5. `ARTTeamPlanningChannel::IsNetRelevantFor` autorizza i client del team.
6. A ogni riconnessione il server invia uno snapshot degli intenti correnti.
7. Alla risoluzione il server elimina o archivia i piani privati.
8. Un test automatico controlla che il client avversario non possieda l'oggetto o il payload.

`COND_OwnerOnly` da solo non risolve il problema, perché un Actor ha normalmente un singolo owner mentre l'informazione deve raggiungere più compagni. Per un quattro contro quattro, un canale replicato per squadra o RPC mirate risultano più semplici da verificare.

UX della pianificazione

La UI deve distinguere tre livelli di informazione:

Livello	Contenuto
Piano proprio	Dettaglio completo, costi, errori, timeline
Piano alleato	Path, destinazione, AoE, target, label e ready state
Nemico	Posizione e stato pubblicamente visibili, mai il piano

Codifica visuale:

Informazione	Forma
Movimento	Linea segmentata con frecce
Cella finale	Contorno pieno
Tiro	Linea sottile con punto target
Area d'effetto	Reticolo o pattern
Difesa	Arco o cupola
Interazione	Icona contestuale
Piano non confermato	Trasparenza e tratteggio
Piano confermato	Forma più stabile e lucchetto
Conflitto	Icona, testo e pattern di avviso

Il colore non deve essere l'unico canale: ogni giocatore riceve anche un numero, una forma e un pattern. Common UI fornisce un'architettura per UI stratificate e input multiplatforma; Enhanced Input consente Mapping Context dinamici, trigger e modificatori, utili per passare tra camera, selezione, targeting e UI. ⁷

IA tattica

Pipeline proposta:

```
Snapshot pubblico
|
v
Celle raggiungibili
|
v
Azioni candidate
|
v
Filtro legalità
|
v
Utility individuale
|
v
Coordinamento di squadra
|
v
Piano scelto
```

Utility:

```
Score =
  ObjectiveValue
+ ExpectedDamage
+ AllySynergy
+ PositionValue
+ CoverValue
- Exposure
- HazardRisk
- ResourceCost
- CollisionRisk
```

L'IA non deve leggere i piani privati dei giocatori umani. Il server deve fornire al planner uno snapshot che esclude tali informazioni, anche se tecnicamente le possiede.

StateTree è adatto agli stati macro, come avanzare, difendere, ritirarsi o perseguire un obiettivo. Behavior Tree e Blackboard sono utili per comportamenti gerarchici, mentre EQS offre un modello per raccogliere e pesare posizioni candidate. Il planner simultaneo specifico deve comunque restare nel core C++, perché deve produrre piani deterministici e coordinati. 8

Stack tecnologico e configurazione del sistema

Requisiti consigliati

Epic indica per Unreal Engine una configurazione raccomandata con **32 GB di RAM**, GPU compatibile DirectX 11 o 12 e circa **8 GB di memoria grafica**. Per un progetto C++ multiplayer con editor, IDE e più client PIE, 32 GB rappresentano il minimo pratico consigliato; 64 GB migliorano la qualità di vita ma non sono un requisito formale. ⁹

Componente	Raccomandazione
Sistema operativo	Windows 11 a 64 bit
CPU	Almeno otto core moderni
RAM	32 GB minimo raccomandato
GPU	DirectX 12, almeno 8 GB VRAM
Disco	SSD NVMe con almeno 200 GB liberi
Motore	Unreal Engine 5.8 stabile
IDE	Visual Studio compatibile con UE 5.8
Version control	Git più Git LFS
Account	Epic Games e GitHub collegati
Testing multiplayer	Due o più processi PIE o standalone

Per UE 5.8, la tabella di compatibilità Epic indica Visual Studio 2022 dalla versione 17.14 e Visual Studio 2026 dalla versione 18.0; la stessa documentazione suggerisce Visual Studio 2026 per lo sviluppo generale, mantenendo Visual Studio 2022 in alcuni toolchain specifici. Verificare sempre la matrice relativa alla versione esatta del motore installata. ¹⁰

Installazione passo passo

Installare Unreal Engine

1. Scaricare e installare [Epic Games Launcher](#).
2. Aprire **Unreal Engine** → **Library**.
3. Aggiungere **Unreal Engine 5.8**.
4. Aprire **Options** prima dell'installazione.
5. Selezionare:
6. Core Components;
7. Templates and Feature Packs;
8. Starter Content, facoltativo;
9. Engine Source, utile per consultazione;
10. Editor Symbols for Debugging, utile ma pesante;
11. Windows target platform.

12. Non usare una build Preview per il branch principale.
13. Avviare l'editor una volta per completare shader e configurazione.

Epic documenta l'installazione tramite Launcher, la selezione dei componenti e il fatto che le build Preview non siano destinate allo sviluppo di produzione. ¹¹

Installare Visual Studio

Nel Visual Studio Installer selezionare:

1. **Game development with C++;**
2. Windows SDK;
3. Visual Studio Tools for Unreal Engine;
4. Visual Studio debugger tools for Unreal Engine Blueprints;
5. Unreal Engine Test Adapter;
6. C++ profiling tools.

Gli strumenti Microsoft integrano visualizzazione delle macro Unreal, log, test e debugging Blueprint/C+++. ¹²

Installare Git e Git LFS

1. Installare Git.
2. Installare Git LFS.
3. Aprire PowerShell.
4. Eseguire:

```
git lfs install
```

1. Nel repository:

```
git lfs track "*.uasset"  
git lfs track "*.umap"  
git lfs track "*.fbx"  
git lfs track "*.wav"  
git add .gitattributes
```

Git LFS richiede l'installazione dell'estensione e il comando iniziale `git lfs install`; i tipi di file vengono poi registrati nel repository. ¹³

`.gitignore` minimo:

```
Binaries/  
DerivedDataCache/  
Intermediate/  
Saved/  
.vs/
```

```
*.VC.db  
*.sln
```

Il file `.uproject`, `Config`, `Content`, `Plugins` e `Source` devono essere versionati.

Collegare Epic e GitHub

L'accesso al repository sorgente Unreal su GitHub richiede il collegamento tra account Epic e GitHub e l'accettazione dell'invito all'organizzazione Epic. ¹⁴

Guida ufficiale:

[Unreal Engine on GitHub](#)

Non è necessario compilare subito il motore dal sorgente. La build del Launcher è più adatta alle prime fasi. La compilazione del motore ha senso quando occorre modificare il motore, diagnosticare problemi profondi o mantenere patch specifiche. Epic fornisce una procedura ufficiale per la build sorgente. ¹⁵

Creazione del progetto

1. Aprire Unreal Engine.
2. Selezionare **Games**.
3. Scegliere **Top Down**.
4. Selezionare **C++**.
5. Target: **Desktop**.
6. Quality preset: **Maximum** inizialmente.
7. Ray tracing: disattivato.
8. Starter Content: facoltativo.
9. Nome: `RefactorTactics`.
10. Percorso senza spazi o caratteri speciali.

Struttura iniziale:

```
RefactorTactics  
├─ Config  
├─ Content  
│   ├─ Core  
│   ├─ Grid  
│   ├─ Characters  
│   ├─ Abilities  
│   ├─ UI  
│   ├─ Maps  
│   └─ Tests  
├─ Plugins  
├─ Source  
│   ├─ RefactorTactics  
│   ├─ RTCore  
│   └─ RTGrid
```

Plugin da attivare progressivamente:

Plugin	Quando
Enhanced Input	Subito
Gameplay Tags	Subito
Gameplay Abilities	Quando iniziano le abilità
Gameplay Tasks	Con Gameplay Ability System
Common UI	Durante la planning UI
Modular Gameplay	Prima delle modalità multiple
Game Features	Prima della vertical slice completa
Functional Testing	Dalla prima regola deterministica
Online Subsystem	Durante la multiplayer alpha
Model-View-ViewModel	Solo se scelto per la UI
Gameplay Message Router	Valutabile per eventi di presentazione

Confronto tra C++, Blueprint e C

Criterio	Unreal C++	Blueprint	C# tramite plugin
Supporto Epic	Completo	Completo	No
Networking nativo	Completo	Buono	Dipende dal plugin
Test unitari	Ottimo	Limitato	Dipende dal bridge
Performance	Alta	Adeguate per molte funzioni	Variabile
Iterazione	Più lenta	Molto rapida	Rapida con hot reload
Diff e merge	Testuale	Più difficile	Testuale
Accesso API nuove	Immediato	Esposte da Epic/C++	Dipende dai binding
Piattaforme	Tutte quelle supportate	Tutte	Dipende dal plugin
Rischio upgrade motore	Gestibile	Gestibile	Maggiore
Uso consigliato	Core	Presentazione e authoring	Tool o esperimenti

UnrealSharp dichiara supporto per UE 5.6–5.8, .NET 10, hot reload e binding generati, ma resta un progetto open source esterno a Epic e il supporto piattaforme non coincide necessariamente con quello completo del motore. **UnrealCLR** integra .NET 6, mentre MonoUE richiede una versione modificata del motore ed espone limitazioni ancora dichiarate dal progetto. Queste soluzioni sono interessanti come esperimento o per tool interni, non come dipendenza centrale di RefactorTactics. ¹⁶

Decisione consigliata:

C++	Regole, dati runtime, rete, resolver, test
Blueprint	UI, VFX, audio, level scripting, configurazione
C#	Eventuali tool isolati, mai requisito per avviare il gioco

Migrazione mentale da C

C# o Unity	Unreal C++
<code>MonoBehaviour</code>	<code>AActor</code> , <code>UActorComponent</code> , <code>UObject</code> , <code>Subsystem</code>
<code>GameObject</code>	<code>AActor</code>
Component	Actor Component o Scene Component
Prefab	Blueprint Class
ScriptableObject	Data Asset
<code>List<T></code>	<code>TArray<T></code>
<code>Dictionary<K,V></code>	<code>TMap<K,V></code>
<code>HashSet<T></code>	<code>TSet<T></code>
Attribute	Macro di reflection
<code>Task</code>	Task UE, async operation o delegate
Event C#	Delegate Unreal
Garbage collector .NET	Garbage collector UObject
NuGet	Moduli, plugin e file <code>.Build.cs</code>
Scene	Level o Map
Singleton	GameInstance Subsystem, World Subsystem o Engine Subsystem

Epic mantiene una guida per sviluppatori provenienti da Unity che mette in relazione Scene e Map, GameObject e Actor, componenti e Blueprint Class. ¹⁷

Regole pratiche:

1. Non creare un UObject con `new`.
2. Non distruggere un UObject con `delete`.
3. Usare `UPROPERTY` o `TObjectPtr` per riferimenti UObject persistenti.
4. Preferire `TArray`, `TMap`, `TSet`, `FString`, `FName` e `FText`.

5. Separare `.h` e `.cpp`.
6. Non mettere ogni funzione in un Actor.
7. Usare classi e struct C++ pure per gli algoritmi.
8. Esporre a Blueprint soltanto API intenzionali.
9. Evitare Tick quando un evento può fare lo stesso lavoro.
10. Compilare spesso, con modifiche piccole.

Primo esempio copia-incolla

Creare `Source/RefactorTactics/Public/Grid/RTGridTypes.h`:

```
#pragma once

#include "CoreMinimal.h"
#include "RTGridTypes.generated.h"

USTRUCT(BlueprintType)
struct FRTGridCellId
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Y = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Layer = 0;

    bool operator==(const FRTGridCellId& Other) const
    {
        return X == Other.X
            && Y == Other.Y
            && Layer == Other.Layer;
    }
};

FORCEINLINE uint32 GetTypeHash(const FRTGridCellId& Cell)
{
    return HashCombine(
        HashCombine(GetTypeHash(Cell.X), GetTypeHash(Cell.Y)),
        GetTypeHash(Cell.Layer)
    );
}
```

Cosa impari:

Elemento	Significato
USTRUCT(BlueprintType)	Rende la struct visibile a Blueprint
GENERATED_BODY()	Aggiunge codice generato da Unreal Header Tool
UPROPERTY	Espone e traccia i campi
FRT	Prefisso del progetto
GetTypeHash	Permette l'uso come chiave in TMap e TSet

Funzione C++ richiamabile da Blueprint

Creare una classe **Blueprint Function Library** dall'Editor:

```
#pragma once

#include "CoreMinimal.h"
#include "Kismet/BlueprintFunctionLibrary.h"
#include "Grid/RTGridTypes.h"
#include "RTGridBlueprintLibrary.generated.h"

UCLASS()
class REFACTORTACTICS_API URTGridBlueprintLibrary
: public UBlueprintFunctionLibrary
{
    GENERATED_BODY()

public:
    UFUNCTION(
        BlueprintPure,
        Category = "RefactorTactics|Grid"
    )
    static int32 ManhattanDistance(
        const FRTGridCellId& A,
        const FRTGridCellId& B)
    {
        return FMath::Abs(A.X - B.X)
            + FMath::Abs(A.Y - B.Y)
            + FMath::Abs(A.Layer - B.Layer);
    }
};
```

Passi:

1. Chiudere l'Editor se la compilazione live crea problemi.
2. Compilare la soluzione.
3. Riaprire l'Editor.
4. Creare un Blueprint Actor.
5. Cercare il nodo `Manhattan Distance`.

6. Creare due `RTGridCellId`.
7. Stampare il risultato.
8. Provare coordinate su layer differenti.

La guida ufficiale C++ Quick Start copre la creazione di classi, compilazione ed esposizione dei tipi Unreal. ¹⁸

Azione pianificata minima

```
#pragma once

#include "CoreMinimal.h"
#include "GameplayTagContainer.h"
#include "Grid/RTGridTypes.h"
#include "RTPlannedAction.generated.h"

UENUM(BlueprintType)
enum class ERTActionKind : uint8
{
    Move,
    Ability,
    Interact,
    Wait
};

USTRUCT(BlueprintType)
struct FRTPlannedAction
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    ERTActionKind Kind = ERTActionKind::Wait;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FRTGridCellId TargetCell;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FGameplayTag AbilityId;
};
```

Nel file `.Build.cs` aggiungere:

```
PublicDependencyModuleNames.AddRange(
    new string[]
    {
        "Core",
        "CoreUObject",
        "Engine",
        "GameplayTags"
    }
);
```

```
}  
);
```

Il file `.Build.cs` usa sintassi C#, ma non contiene gameplay C#: descrive dipendenze e configurazione del modulo Unreal.

Sample ufficiali consigliati

Risorsa	Utilità
Lyra Sample Game	Architettura modulare, multiplayer, GAS e Common UI
Action RPG Sample	Gameplay Ability System e interazione Blueprint/C++
Content Examples	Esempi isolati delle funzionalità engine
C++ Quick Start	Primo progetto C++
Blueprint Visual Scripting	Fondamenti Blueprint
Epic GitHub	Codice sorgente del motore

Lyra è una risorsa particolarmente utile per modularità, multiplayer e Gameplay Ability System, ma non deve essere copiato indiscriminatamente. Epic segnala inoltre che la migrazione di un progetto Lyra fra major release può richiedere confronto manuale del codice e delle release note. ¹⁹

Curriculum tutorial

Il curriculum produce il prodotto reale. Ogni lezione termina con un deliverable integrato nel repository.

Le durate rappresentano lavoro attivo e non includono eventuali blocchi, approfondimenti o rifacimenti.

Sequenza completa

Lezione	Durata	Obiettivo	Deliverable
Ambiente e repository	Tre ore	Installare tool, creare progetto e Git	Build iniziale versionata
Orientamento nell'Editor	Tre ore	Viewport, Content Browser, Level, PIE e log	Stanza di test
Blueprint essenziali	Quattro ore	Eventi, funzioni, struct e debug	Pedina cliccabile
C++ per chi conosce C#	Cinque ore	Reflection, header, source e UObject	Actor C++ esposto
Enhanced Input	Tre ore	Pan, zoom, click e mapping context	Camera completa
Coordinate della griglia	Cinque ore	World-grid e grid-world	Griglia logica

Lezione	Durata	Obiettivo	Deliverable
Rendering della griglia	Cinque ore	Instanced mesh e highlight	Mappa fluida
Selezione delle celle	Quattro ore	Hover, click e feedback	Selettore riutilizzabile
Grafo multilivello	Sei ore	Nodi, archi e layer	Due piani collegati
A* minimale	Sei ore	Open set, costi e parent	Percorso deterministico
Cost provider	Sei ore	Terreno, profilo e pericolo	Percorsi diversi per unità
Linea di vista	Sei ore	Altezza, cover e tracing	Preview tiro
Turn manager	Sei ore	Planning, lock e aftermath	Round locale
Piano di azioni	Sei ore	Movimento, abilità e undo	Timeline del piano
Resolver simultaneo	Otto ore	Microstep, conflitti e priorità	Due unità risolte
Ability System	Otto ore	Ability, effect, tag e cooldown	Tre abilità
Planning UI	Sei ore	UMG, Common UI e tooltip	HUD funzionale
Intenti condivisi locali	Cinque ore	Ghost, path, AoE e label	Due piani visibili
Networking fondamentale	Otto ore	GameMode, GameState, RPC e replica	Match listen server
Privacy degli intenti	Otto ore	Replica filtrata e anti-leak	Alleati sì, nemici no
IA tattica	Dieci ore	Candidate generation e utility	Bot con piano valido
Personalizzazione	Otto ore	Data Asset, moduli e budget	Due build differenti
Test e analytics	Otto ore	Automation, Insights ed eventi	Suite automatica
Packaging	Otto ore	Development, Shipping e config	Demo installabile
Mod dati	Dieci ore	JSON, schema e validator	Nuovo terreno senza build
Roguelike	Sedici ore	Run, deck, ricompense e save	Mini-run cooperativa

Criteri di completamento di ogni lezione

Una lezione è conclusa soltanto quando:

1. il progetto compila da una checkout pulita;
2. il comportamento è riproducibile;
3. esiste almeno un test per la nuova regola;
4. il deliverable è usabile dalla lezione successiva;
5. non sono presenti errori rossi nel log;

6. il codice è committato;
7. la documentazione del modulo è aggiornata;
8. le decisioni temporanee sono registrate come debito tecnico.

Milestone didattiche

Milestone	Cosa sai fare
Sandbox interattiva	Editor, Blueprint, input e Actor
Griglia funzionante	Struct, container, conversioni e rendering
Pathfinding multilivello	Algoritmi C++, grafi e test
Turno locale	Macchina a stati e separazione modello-vista
Ability vertical slice	GAS, tag, effetti e dati
Planning collaborativo	UI complessa e visualizzazione
Multiplayer privato	RPC, autorità e replica selettiva
Beta tattica	IA, personalizzazione e bilanciamento
Build distribuibile	Packaging, profiling e crash handling
Modding	JSON, schema, dipendenze e sicurezza
Espansione roguelike	Save, generazione run e deckbuilding

Metodo di studio consigliato

Per ogni lezione:

```

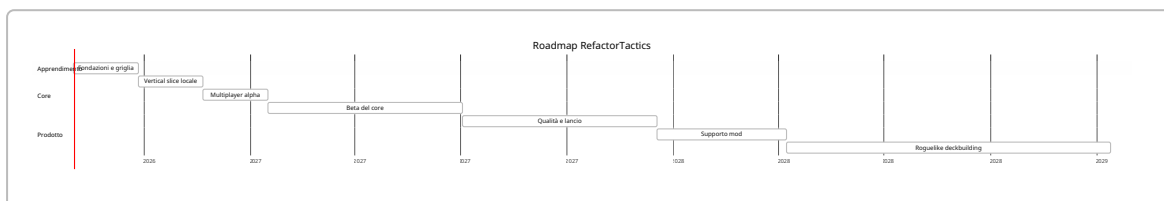
Leggi il concetto
  |
  v
Riproduci l'esempio minimo
  |
  v
Scrivi un test
  |
  v
Integra nel gioco
  |
  v
Rompi volontariamente il sistema
  |
  v
Leggi log e debugger
  |
  v
Rifattorizza

```

Non copiare grandi sistemi da Lyra all'inizio. Leggi Lyra quando hai già costruito una versione semplice dello stesso problema: a quel punto il sample diventa comprensibile invece di sembrare magia nera.

Roadmap dal tutorial al prodotto

Timeline di alto livello



Versione leggibile in stampa:

Fondazioni e griglia	Mesi iniziali	Sandbox interattiva
Vertical slice locale	Fase successiva	Match locale completo
Multiplayer alpha	Dopo la slice	Due contro due online
Beta del core	Fase centrale	Quattro contro quattro
Qualità e lancio	Pre-release	Prodotto PC
Supporto mod	Dopo il core	SDK dati
Roguelike deckbuilding	Espansione	Co-op da uno a quattro

Fasi e gate

Fase	Contenuto	Gate di uscita
Fondazioni didattiche	Editor, Git, Blueprint, C++, camera	Sandbox avviabile
Griglia	Coordinate, rendering, selezione	Mappa interattiva
Pathfinding	Grafo, layer, cost provider	Percorsi verificati
Vertical slice locale	Turni, abilità, UI e ambiente	Match uno contro uno
Multiplayer alpha	Autorità, intenti e riconnessione	Due contro due stabile
Core beta	Quattro contro quattro, IA, build	Feature complete
Qualità	Accessibilità, performance e replay	Release candidate
Lancio	Packaging, servizi e operazioni	Prodotto PC
Mod support	Loader, schema, validator e browser	SDK mod
Roguelike	PvE, run, deck, boss e save	Espansione co-op

Scope della vertical slice

La vertical slice deve contenere solo:

Sistema	Quantità
Mappa	Una
Layer	Due
Personaggi	Quattro
Abilità per personaggio	Tre
Specializzazioni	Nessuna o una semplificata
Modalità	Relay Control
Giocatori	Uno contro uno, bot opzionale
Tipi di terreno	Quattro
Collegamenti verticali	Scala e jump pad
Effetti ambientali	Fuoco e acqua
Interazioni	Porta e terminale
UI planning	Completa ma placeholder
Networking	Escluso dalla prima slice

La slice deve dimostrare:

1. movimento multilivello;
2. un percorso diverso per due unità;
3. una abilità che modifica la mappa;
4. una interazione fra superficie e abilità;
5. un conflitto simultaneo;
6. una preview leggibile;
7. una risoluzione ripetibile.

Alpha multiplayer

Aggiunge:

Area	Requisito
Server	Autorevole
Match	Due contro due
Intention sharing	Solo alleati
Reconnect	Snapshot del round
Ready state	Replicato
Timeout	Gestito server-side
Abbandono	Bot o surrender

Area	Requisito
Logging	Piani, eventi e hash
Test	Due processi più packet simulation

Il sistema online può essere costruito dietro un'interfaccia. Common User è usato in Lyra per login, autenticazione e sessioni, mentre l'integrazione EOS richiede registrazione e configurazione del prodotto. ²⁰

Beta del core

La beta è feature-complete quando comprende:

- quattro contro quattro;
- almeno otto personaggi;
- almeno tre mappe;
- una modalità principale;
- personalizzazione orizzontale;
- bot capaci di completare i match;
- tutorial giocabile;
- replay tecnico;
- analytics;
- riconnessione;
- accessibilità di base;
- validator mappe;
- suite deterministica;
- build installabile.

Espansione cooperativa roguelike

La modalità roguelike non sostituisce il prodotto core: usa la stessa griglia, lo stesso resolver, le stesse abilità e lo stesso networking.

Loop:

```

Scegli personaggio e starter deck
    |
    v
Scegli un nodo della run
    |
    v
Combattimento simultaneo
    |
    v
Ricompensa o evento
    |
    v
Modifica deck e build
    |
    v

```



Struttura proposta:

Elemento	Baseline
Giocatori	Da uno a quattro
Atti	Tre
Combattimenti	Da sei a dieci
Miniboss	Uno per atto
Boss finale	Uno
Deck iniziale	Otto-dodici carte
Mano	Quattro-sei carte
Energia	Risorsa rinnovata per round
Reliquie	Passive della run
Eventi	Scelte narrative e meccaniche
Salvataggio	Tra nodi, con versione schema
Meta-progression	Sblocchi orizzontali

Decisione critica: **movimento base, attacco base e difesa minima devono essere sempre disponibili**, indipendentemente dalla mano. Le carte aggiungono opzioni, modificano abilità o creano combo; non devono poter produrre un round in cui il giocatore non può fare nulla.

Categorie di carte:

Categoria	Esempio
Azione	Colpo perforante
Movimento	Salto termico
Reazione	Scudo di emergenza
Modifica	La prossima abilità crea ghiaccio
Coordinazione	Un alleato può seguire il tuo percorso
Ambiente	Elettrifica le celle bagnate
Evocazione	Posiziona un drone

Categoria	Esempio
Manipolazione deck	Pesca, scarta, conserva
Rischio	Più potenza ma genera pericolo

Generazione delle mappe:

```

Stanze progettate a mano
+
Parametri semantici
+
Regole di connessione
+
Validator automatico
=
Incontro procedurale controllato

```

Non generare liberamente ogni cella nelle prime versioni. Assemblare stanze curate riduce mappe irrisolvibili e mantiene leggibilità tattica.

Modding, analytics, test e rischi

Strategia delle mod

Il supporto mod è l'ultima grande feature del prodotto core, ma le decisioni necessarie vengono prese sin dall'inizio:

1. ogni contenuto ha un ID namespaced;
2. i dati hanno una versione schema;
3. i riferimenti non dipendono da path fragili;
4. regole e presentazione sono separate;
5. le operazioni sono dichiarative;
6. il loader produce errori leggibili;
7. client e server confrontano il manifest;
8. il resolver non carica codice arbitrario.

Livelli di modding

Livello	Contenuto	Matchmaking pubblico
Data mod	JSON per terreno, carte, effetti e incontri	Sì, se validata
Content mod	Mesh, audio, materiali e mappe cotte	Solo con stesso manifest
Game Feature mod	Modalità o pacchetto complesso	Server dedicati o allowlist
Native mod	Codice C++	No, salvo ambienti privati

Il sistema Game Features offre una base utile per pacchetti di contenuto modulari, mentre Primary Data Asset e Asset Manager permettono caricamento e scaricamento controllato dei dati. ²¹

Manifest di una mod

```
{
  "schemaVersion": 1,
  "namespace": "aaron.volcanic_pack",
  "modId": "volcanic_pack",
  "displayName": "Volcanic Pack",
  "version": "1.0.0",
  "engineRange": {
    "minimum": "5.8.0",
    "maximumExclusive": "5.9.0"
  },
  "gameVersionRange": {
    "minimum": "1.0.0",
    "maximumExclusive": "2.0.0"
  },
  "dependencies": [
    {
      "modId": "core",
      "minimumVersion": "1.0.0"
    }
  ],
  "content": [
    "terrain.lava_crust",
    "effect.burning",
    "card.thermal_jump"
  ],
  "network": {
    "deterministic": true,
    "rankedAllowed": false
  }
}
```

Terreno moddabile

```
{
  "schemaVersion": 1,
  "namespace": "aaron.volcanic_pack",
  "id": "terrain.lava_crust",
  "type": "terrain",
  "displayName": "Crosta lavica",

  "tags": [
    "Terrain.Hot",
    "Terrain.Unstable"
  ],

  "movement": {
    "baseCost": 3,
```

```

    "blockedBy": [
      "Unit.Tag.Fragile"
    ],

    "costModifiers": [
      {
        "whenUnitHas": "Unit.Tag.Fireproof",
        "add": -2
      }
    ]
  },

  "onEnter": [
    {
      "effect": "effect.burning",
      "stacks": 1
    }
  ],

  "network": {
    "deterministic": true
  },

  "limits": {
    "maxInstancesPerMap": 300
  }
}

```

Carta roguelike moddabile

```

{
  "schemaVersion": 1,
  "namespace": "core",
  "id": "card.thermal_jump",
  "type": "roguelike_card",
  "displayName": "Salto termico",
  "rarity": "uncommon",
  "energyCost": 1,

  "requiresTags": [
    "Character.Mobility"
  ],

  "targeting": {
    "kind": "cell",
    "range": 5
  },

  "effects": [

```

```

    {
      "op": "addTraversalEdge",
      "edgeType": "Jump",
      "durationTurns": 1
    },
    {
      "op": "applyStatus",
      "status": "status.evasion",
      "durationTurns": 1
    }
  ]
}

```

Operazioni autorizzate

Invece di eseguire script arbitrari, la prima API pubblica deve offrire un catalogo di operazioni:

```

applyDamage
applyHealing
applyStatus
removeStatus
moveUnit
pushUnit
pullUnit
spawnEntity
destroyEntity
modifyCellTags
addTraversalEdge
removeTraversalEdge
createHazard
drawCards
discardCards
gainEnergy
registerTrigger

```

Ogni operazione ha:

- schema noto;
- range numerici;
- numero massimo di target;
- ordine di esecuzione;
- comportamento deterministico;
- regole di rete;
- versione.

Handshake multiplayer

```

SERVER
  crea lista ordinata mod

```



Le partite classificate devono usare esclusivamente:

- contenuti core;
- mod firmate;
- allowlist server;
- schema approvato;
- stesso hash;
- nessun codice nativo esterno.

Analytics hooks

Unreal offre un'interfaccia analytics astratta con provider intercambiabili; esiste anche un provider di file utile per sviluppo e test che scrive eventi JSON, ma non è pensato come backend di produzione. ²²

Eventi minimi:

Evento	Dati non personali
<code>match_started</code>	modalità, mappa, build version
<code>planning_started</code>	round e durata timer
<code>plan_changed</code>	revisione e tipo modifica
<code>intent_viewed</code>	tipo intento alleato
<code>plan_locked</code>	tempo residuo
<code>ally_conflict_warned</code>	tipo conflitto
<code>path_preview_completed</code>	nodi visitati e tempo
<code>resolution_completed</code>	durata e numero eventi
<code>desync_detected</code>	fase e hash
<code>ability_used</code>	ability ID e contesto
<code>build_selected</code>	moduli, senza identità personale
<code>match_ended</code>	risultato e durata

Evento	Dati non personali
card_drafted	carta e alternative
run_node_entered	tipo nodo e atto
mod_validation_failed	codice errore e schema

Gli analytics devono passare attraverso un adapter interno:

```
class IRTAnalyticsSink
{
public:
    virtual ~IRTAnalyticsSink() = default;

    virtual void RecordEvent(
        const FString& EventName,
        const TMap<FString, FString>& Attributes) = 0;
};
```

Il gameplay non deve conoscere il provider esterno.

Test plan

Test unitari

Sistema	Test
Coordinate	Conversione world-grid
Grafo	Vicini e archi
A*	Percorso minimo e nessun percorso
Cost provider	Immunità, blocchi e modificatori
LOS	Ostacoli, altezza e cover
Resolver	Ordine eventi
Movimento	Collisioni e swap
Status	Stack e durata
Mod loader	Schema e dipendenze
Save	Versione e migrazione
Deck	Pesca, scarto e reshuffle

Test deterministici

Per ogni scenario:


```
Load fixture
Apply seed
Submit plans
Resolve phase
Hash state
Compare golden hash
```

Testare:

- stesso risultato in cento esecuzioni;
- ordine diverso di inserimento degli Actor;
- frame rate differente;
- server headless;
- replay da log;
- mod caricate in ordine differente ma normalizzato.

Test di rete

Caso	Verifica
Latenza alta	Submit e conferma coerenti
Packet loss	Nessun piano perso definitivamente
Reconnect	Snapshot corretto
Client malevolo	Piano illegale rifiutato
Enemy inspection	Nessun intento privato presente
Timeout	Server applica fallback
Version mismatch	Connessione rifiutata chiaramente
Mod mismatch	Differenze elencate
Host migration	Fuori scope iniziale o formalmente gestita

Networking Insights può analizzare il traffico di rete e le proprietà replicate, mentre Unreal Insights raccoglie dati di tracing e profiling ad alta frequenza. ²³

Test funzionali

Unreal supporta Functional Test Actor eseguibili nei livelli; l'Unreal Test Adapter di Visual Studio consente inoltre di individuare, eseguire e debuggare test Unreal dall'IDE. ²⁴

Mappe test dedicate:

```
Test_Pathfinding
Test_Multilayer
Test_LineOfSight
Test_Cover
```

```
Test_SimultaneousCollision
Test_TeamIntentPrivacy
Test_EnvironmentInteractions
Test_ModValidation
Test_RoguelikeSave
```

Crash e osservabilità

Crash Reporter può essere incluso nelle build pacchettizzate e configurato per inviare report a un endpoint controllato dal progetto. ²⁵

Log categories:

```
DECLARE_LOG_CATEGORY_EXTERN(LogRTGrid, Log, All);
DECLARE_LOG_CATEGORY_EXTERN(LogRTTurn, Log, All);
DECLARE_LOG_CATEGORY_EXTERN(LogRTNetwork, Log, All);
DECLARE_LOG_CATEGORY_EXTERN(LogRTMod, Log, All);
```

Ogni round deve poter generare un log compatto:

```
MatchId
Round
MapRevision
Seed
PlanHashes
ResolvedEventHashes
FinalStateHash
```

Gate di release

La release candidate non può procedere se esiste uno dei seguenti problemi:

Gate	Requisito
Crash	Nessun blocker riproducibile
Privacy	Nessun leak del piano nemico
Determinismo	Nessun desync nella suite golden
Networking	Riconnessione validata
Accessibilità	Input rimappabile e informazioni non solo cromatiche
Tutorial	Completamento end-to-end
Performance	Budget rispettati sulle macchine target
Modding	Errori schema non causano crash
Salvataggi	Migrazioni testate

Gate	Requisito
Packaging	Build Shipping installabile
Analytics	Eventi verificati e disattivabili secondo configurazione
Crash reporting	Pipeline testata

Rischi principali

Rischio	Impatto	Mitigazione
Scope troppo ampio	Alto	Vertical slice piccola e feature gate
Curva C++	Medio	Curriculum incrementale e API Blueprint
Desync	Critico	Server authority, interi, seed e hash
Leak dei piani	Critico	Replica per team e audit automatici
Troppe entità per cella	Alto	Strutture dati chunked e instancing
Blueprint ingestibili	Alto	Regole in C++ e grafi piccoli
GAS troppo complesso presto	Medio	Abilità manuali prima, migrazione controllata
Bilanciamento combinatorio	Alto	Budget build, bot e analytics
IA apparentemente sleale	Alto	Snapshot senza piani umani
Mod malevole	Alto	API dichiarativa e nessun native code pubblico
Aggiornamento Unreal	Medio	Version pinning e branch upgrade
Dipendenza plugin C#	Alto	Plugin opzionale, core C++ indipendente
Procedurale incoerente	Alto	Stanze curate e validator
Roguelike troppo casuale	Medio	Azioni base sempre disponibili
Costi asset	Alto	Placeholder fino alla beta core

Decisione architetturale conclusiva

Il cuore di RefactorTactics deve poter funzionare senza grafica:

```

Snapshot iniziale
+
Piani dei giocatori
+
Seed
=
Eventi risolti
+
Snapshot finale

```

+
Hash

Unreal Engine fornisce networking, UI, asset management, Gameplay Ability System, modular gameplay, strumenti IA, profiling e packaging. Tuttavia, **le regole tattiche, il grafo semantico e il resolver devono appartenere al progetto**, non essere nascosti dentro Actor, animazioni o Blueprint di livello.

Questa impostazione permette di usare lo stesso core per:

PvP competitivo
Bot e training
Replay
Editor mappe
Test automatici
Server dedicato
Mod
Co-op roguelike
Deckbuilding

È il punto che rende RefactorTactics non soltanto un gioco ispirato ad Atlas Reactor, ma una piattaforma tattica coerente, studiabile e realmente espandibile.

1 Unreal Engine 5.8 is now available

https://www.unrealengine.com/news/unreal-engine-5-8-is-now-available?utm_source=chatgpt.com

2 Blueprint Overview | Unreal Engine 4.27 Documentation

https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-overview?application_version=4.27&utm_source=chatgpt.com

3 Gameplay Ability System | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27&utm_source=chatgpt.com

4 <https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine>

<https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine>

5 <https://dev.epicgames.com/documentation/unreal-engine/game-features-and-modular-gameplay-in-unreal-engine>

<https://dev.epicgames.com/documentation/unreal-engine/game-features-and-modular-gameplay-in-unreal-engine>

6 Replicating Variables in Blueprints | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/replicating-variables-in-blueprints?application_version=4.27&utm_source=chatgpt.com

7 https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-for-unreal-engine?application_version=5.2

https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-for-unreal-engine?application_version=5.2

8 <https://dev.epicgames.com/documentation/unreal-engine/behavior-trees-in-unreal-engine>

<https://dev.epicgames.com/documentation/unreal-engine/behavior-trees-in-unreal-engine>

- 9 <https://dev.epicgames.com/documentation/unreal-engine/hardware-and-software-specifications-for-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/hardware-and-software-specifications-for-unreal-engine>
- 10 <https://dev.epicgames.com/documentation/unreal-engine/setting-up-visual-studio-development-environment-for-cplusplus-projects-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/setting-up-visual-studio-development-environment-for-cplusplus-projects-in-unreal-engine>
- 11 <https://dev.epicgames.com/documentation/unreal-engine/install-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/install-unreal-engine>
- 12 <https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install>
<https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install>
- 13 <https://git-lfs.com/>
<https://git-lfs.com/>
- 14 <https://www.unrealengine.com/ue-on-github>
<https://www.unrealengine.com/ue-on-github>
- 15 <https://dev.epicgames.com/documentation/unreal-engine/building-unreal-engine-from-source>
<https://dev.epicgames.com/documentation/unreal-engine/building-unreal-engine-from-source>
- 16 <https://github.com/UnrealSharp/UnrealSharp>
<https://github.com/UnrealSharp/UnrealSharp>
- 17 https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27
- 18 <https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-cpp-quick-start>
<https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-cpp-quick-start>
- 19 <https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine>
- 20 <https://dev.epicgames.com/documentation/unreal-engine/using-lyra-with-epic-online-services-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/using-lyra-with-epic-online-services-in-unreal-engine>
- 21 [unreal.PrimaryDataAsset](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PrimaryDataAsset?application_version=5.4&utm_source=chatgpt.com)
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PrimaryDataAsset?application_version=5.4&utm_source=chatgpt.com
- 22 <https://dev.epicgames.com/documentation/unreal-engine/in-game-analytics-for-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/in-game-analytics-for-unreal-engine>
- 23 <https://dev.epicgames.com/documentation/unreal-engine/unreal-insights-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/unreal-insights-in-unreal-engine>
- 24 [unreal.FunctionalTest](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/FunctionalTest?application_version=5.0&utm_source=chatgpt.com)
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/FunctionalTest?application_version=5.0&utm_source=chatgpt.com
- 25 <https://dev.epicgames.com/documentation/unreal-engine/crash-reporting-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/crash-reporting-in-unreal-engine>